



Developing Finite Element Analysis Applications using Python

ASEP Webinar Series 2026-2027 W01

Engr. Jaydee N. Lucero

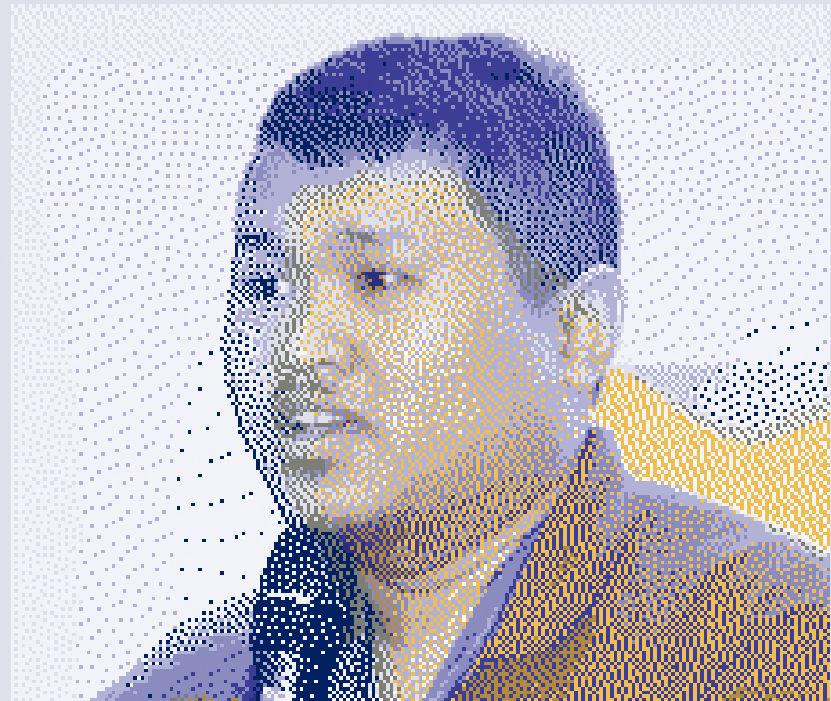
Senior Structural Engineer I

Abinales Associates Engineers + Consultants



Hi! I'm Jaydee.

- * *Senior Structural Engineer I* (2021-present), Abinales Associates Engineers + Consultants
- * *Part-time Review Instructor* (2019-present), Review Innovations
- * *Associate Member* (2022-present), Association of Structural Engineers of the Philippines, Inc.
- * *MS Civil Engineering (Structural Engineering) student* (2024-present), University of the Philippines Diliman
- * Pythonista since August 2024



Outline

1. Introduction
2. Finite element analysis concepts
3. Python libraries from scratch
4. Example 1 A two-dimensional truss analysis program
5. Python FEA libraries
6. Example 2 A two-dimensional isolated foundation design program
7. Next steps



QR Codes

Python 3.14 cheat sheet



PDF and source code of presentation



Part 1

Introduction

A quick recap



We can make our structural engineering programs even more sophisticated.



Why Python for FEA?

Compared to black-box FEA software, an FEA program created using programming languages like Python can be

- * Customizable

Inspect and modify *every* line

- * Open-source

No fees, math fully visible

- * Parametric

Change a variable, rerun, done

- * Ecosystem

numpy, scipy, matplotlib, openseespy, pyniteFEA, etc.

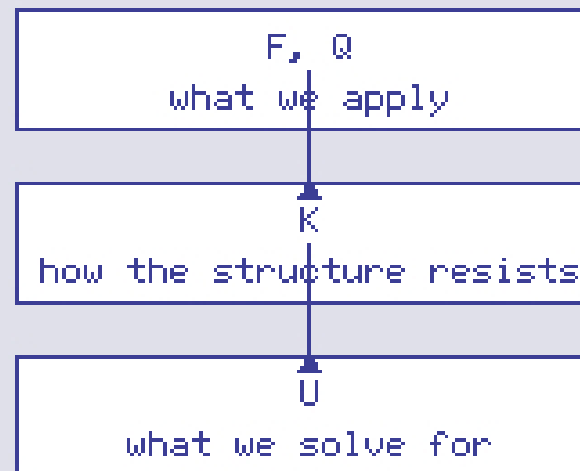
Aspect	GUI	Python
Cost	Fee	Free
Customize	Fixed	Unlimited
Automation	Macro	Native
Parametric	Clicks	Variable
Source	Closed	Open



Demystifying the black box

Every commercial FEA program reduces to just one equation.

$$KU = F + Q$$



In this talk, we will build FEA programs in two ways:

- * from scratch using basic Python libraries, and
- * from Python FEA libraries



Part 2

Finite element analysis concepts

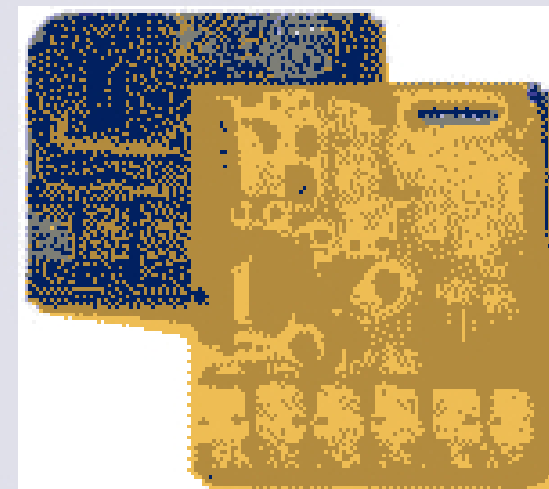
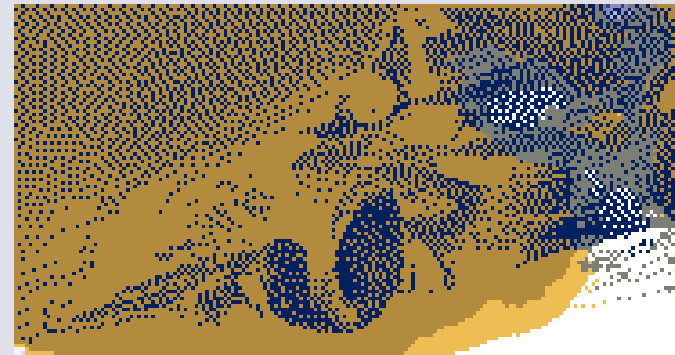
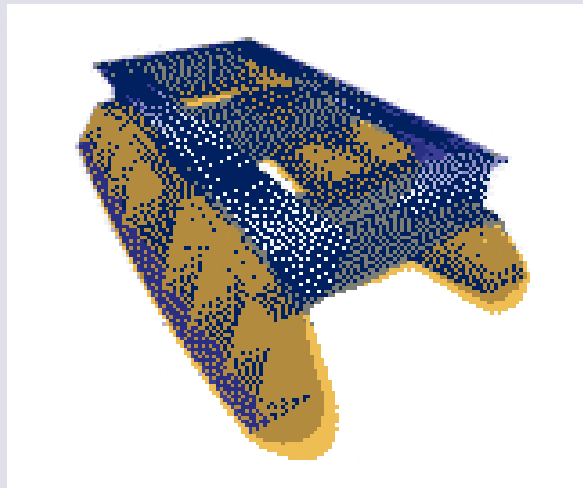
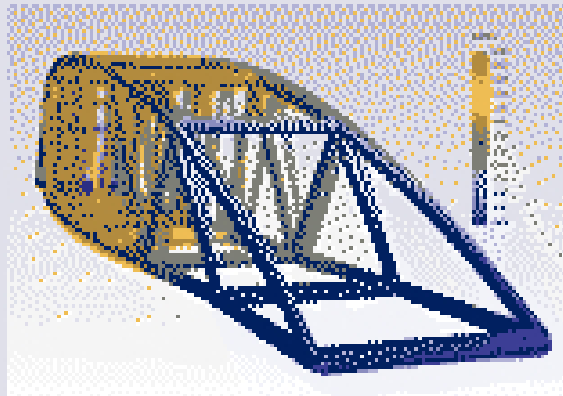
What is finite element analysis?

Instead of providing the infinite number of solutions as in the exact solutions, the **finite element concept** is to determine solutions only at some finite locations. This is done by first discretizing the geometry of the model into a number of finite elements... These elements are connected at grid points or nodes at which the unknowns are to be determined (Dechaumpai, 2024).

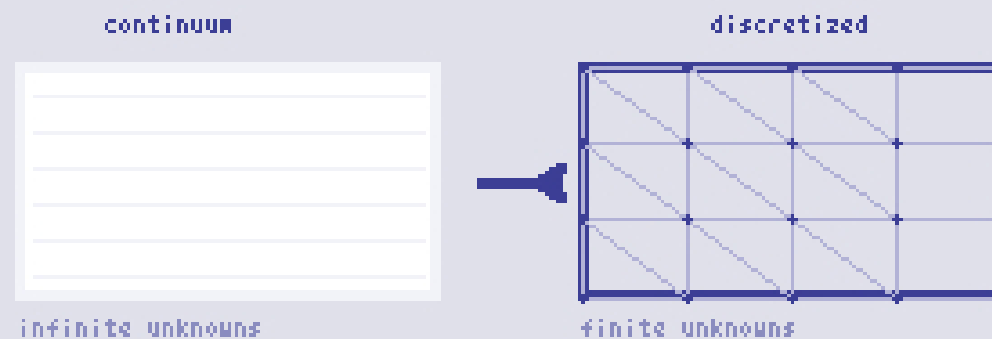
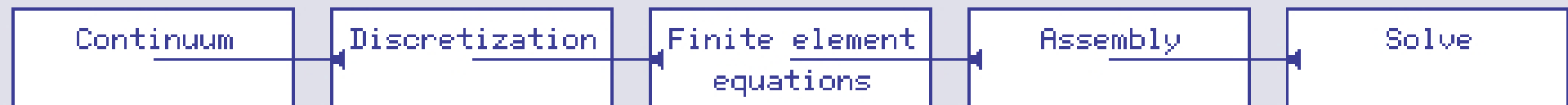
The key idea of **finite element method** is to transform the differential equations into a set of algebraic equations for each element. The finite element equations from all elements are then assembled together to form a large set of simultaneous equations. The boundary conditions of the problem are applied prior to solving for the unknowns at all nodes (Dechaumpai, 2024).



What is finite element analysis?

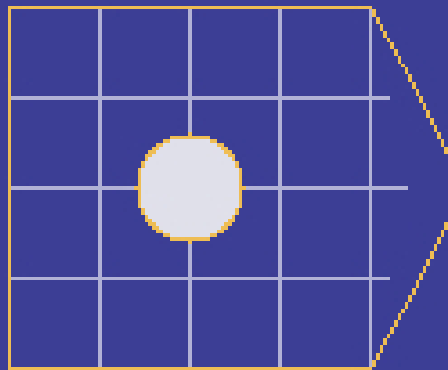


What is finite element analysis?

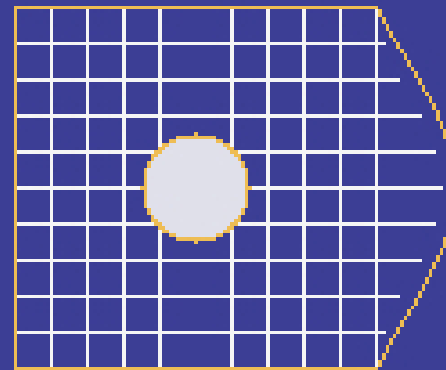


What is finite element analysis?

- * Smaller elements -> more accurate, but more elements -> more computational time
- * Larger elements -> less elements, but less accuracy -> less computational time



Coarse Mesh



Fine Mesh



Some terms

Mathematical model

- * A differential equation or a set of differential equations that mimic the behavior of a natural or physical system.

Domain

- * Everything that is in the system, including geometry, loads, constraints.

Node

- * A point in the domain.

Element

- * A group of adjacent nodes that are related to each other by the mathematical model.

Discretization

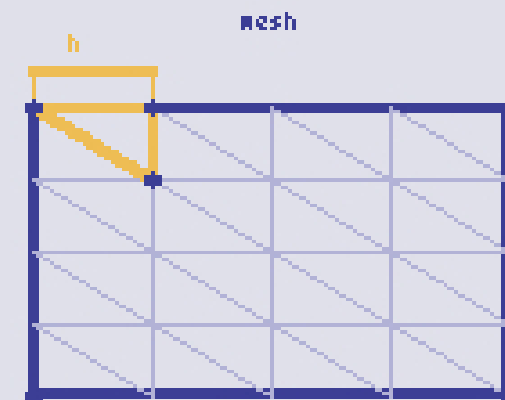
- * The process of dividing the domain into nodes and elements.

Finite element mesh

- * The result of discretization, a collection of nodes and elements approximating the domain.

Element size

- * Distance between two nodes in an element, denoted by h .



Some terms

Boundary

- * Edge (2D) or surface (3D) of the domain.

Boundary conditions

- * Loads and constraints that occur at the boundary.

Finite element equation

- * A matrix equation derived from the integral formulation of a mathematical model.
- * Takes the form $\mathbf{k}\mathbf{u} = \mathbf{f} + \mathbf{q}$. Small letters denote that it is for an element. Here,
 - * $\mathbf{k}$ = stiffness matrix
 - * $\mathbf{u}$ = displacements vector
 - * $\mathbf{f}$ = point load vector
 - * $\mathbf{q}$ = distributed load vector

Assembly

- * Combining the finite element equations from all elements into a single finite element equation for the domain.
- * Takes the form $\mathbf{K}\mathbf{U} = \mathbf{F} + \mathbf{Q}$. The capital letters denote that it is for the whole structure.

global K (6 x 6)

k2					
	k1				
			k3		

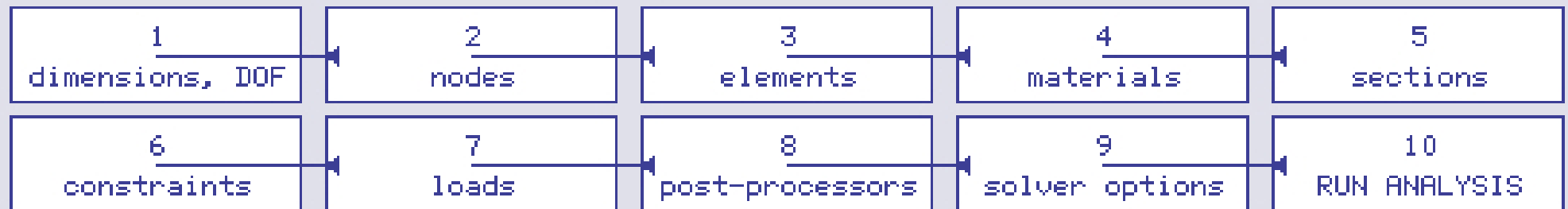


Part 3

Python FEA from scratch

Basic Python FEA concepts

General procedure



Nodes

Nodes can be represented as a list/tuple,

```
1 coords = [(0, 0, 0), (0, 6, 0), (6, 0, 0), (2, 2, 4)]
```

...or as a dictionary,

```
1 coords = {"P1": (0, 0, 0), "P2": (0, 6, 0), "P3": (6, 0, 0), "P4": (2, 2, 4)}
```

...or as an instance of a class.

```
1 class Node:
2     def __init__(self, x, y, z, id) :
3         self.x, self.y, self.z, self.id = x, y, z, id
4
5 P1 = Node(0, 0, 0, "P1")
```



Basic Python FEA concepts

Elements

Depending on your representation of nodes, your representation of elements will differ.

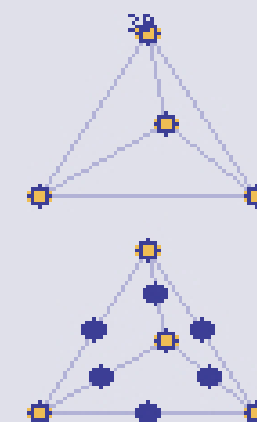
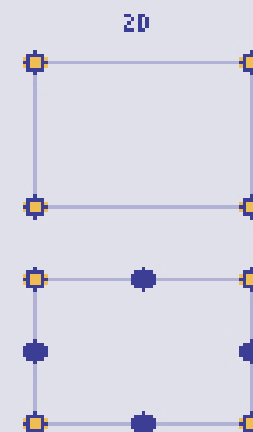
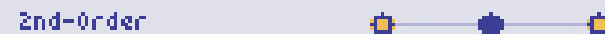
```
1 # Assuming using two-node bar elements.
2
3 # as list/tuples
4 elements = [(0, 3), (1, 3), (2, 3)]
5
6 # as dictionaries
7 elements = {"E1": ("P1", "P4"), "E2": ("P2", "P4"), "E3": ("P3", "P4")}
8
9 # as class instances
10 class Element:
11     def __init__(self, start, end):
12         self.start, self.end = start, end
13
14 E1 = Element(P1, P4)
```



Basic Python FEA concepts

Element finite element equations

- * To estimate the response of the system between nodes, interpolation functions are used.
- * **First-order elements** are elements that use linear functions as interpolating functions.
- * **Second-order elements** are elements that use quadratic functions as interpolating functions.



- * Refer to FEA books (e.g. Reddy) on per-element finite element equations in matrix form



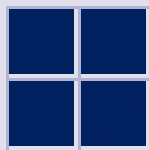
Basic Python FEA concepts

Assembly

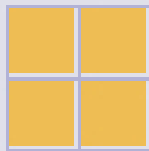
Direct Stiffness Assembly Mapping



Element 1 (K^1)



Element 2 (K^2)



Global Stiffness Matrix (K^6)

	1	2	3	4
1	Dark Blue	Dark Blue	White	White
2	Dark Blue	Superposition (Dark Blue)	Orange	White
3	White	Orange	Orange	White
4	White	White	White	White

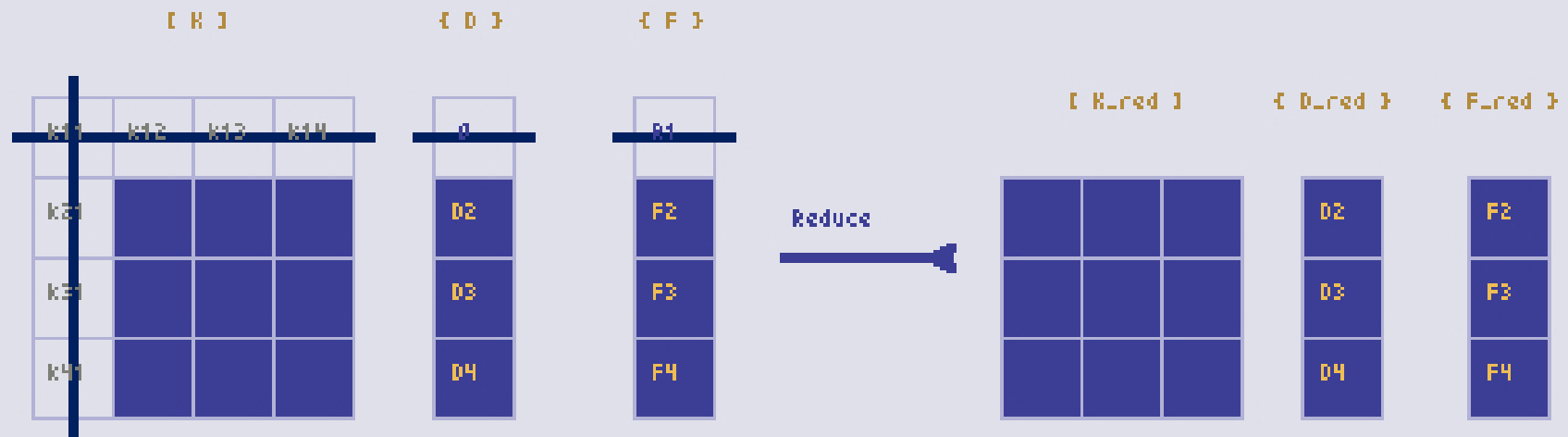
- Element 1 DOF Map
- Element 2 DOF Map
- Superposition at Node 2 (Addition)



Basic Python FEA concepts

Applying boundary conditions

Applying Boundary Conditions (Direct Elimination)



Basic Python FEA concepts

Python libraries

* `numpy` for matrix creation and operations. Install using `pip install numpy`.

```
1 import numpy as np          # Import the numpy library.
2
3 k1, k2 = 4000, 6000
4 K = [[ k1,    -k1,    0],
5       [-k1, k1 + k2, -k2],
6       [ 0,    -k2,   k2]]
7 K = np.array(K, dtype='float') # Create the numpy array.
8
9 f1, f2 = 3000, 5000
10 F = [f1, f1 + f2, f2]
11 F = np.array(F, dtype='float').T # Create the numpy array, then transpose.
12
13 U = np.linalg.inv(K) @ F        # Solve the system [K]{U} = {F} using matrix inverse method
14 print(U)                       # {U} = [K]-1 {F}.
```



Basic Python FEA concepts

Python libraries

* `scipy` for faster matrix operations, especially for sparse arrays.

* A sparse array is a type of matrix in which most of the entries are zero.

Example from here

```
1 import numpy as np
2 from scipy.sparse import csr_array          # Import the scipy library.
3
4 A = csr_array([[1, 2, 0], [0, 0, 3], [4, 0, 5]]) # Create a compressed sparse row array.
5 v = np.array([1, 0, -1])
6 u = A @ v
7 print(u)
```

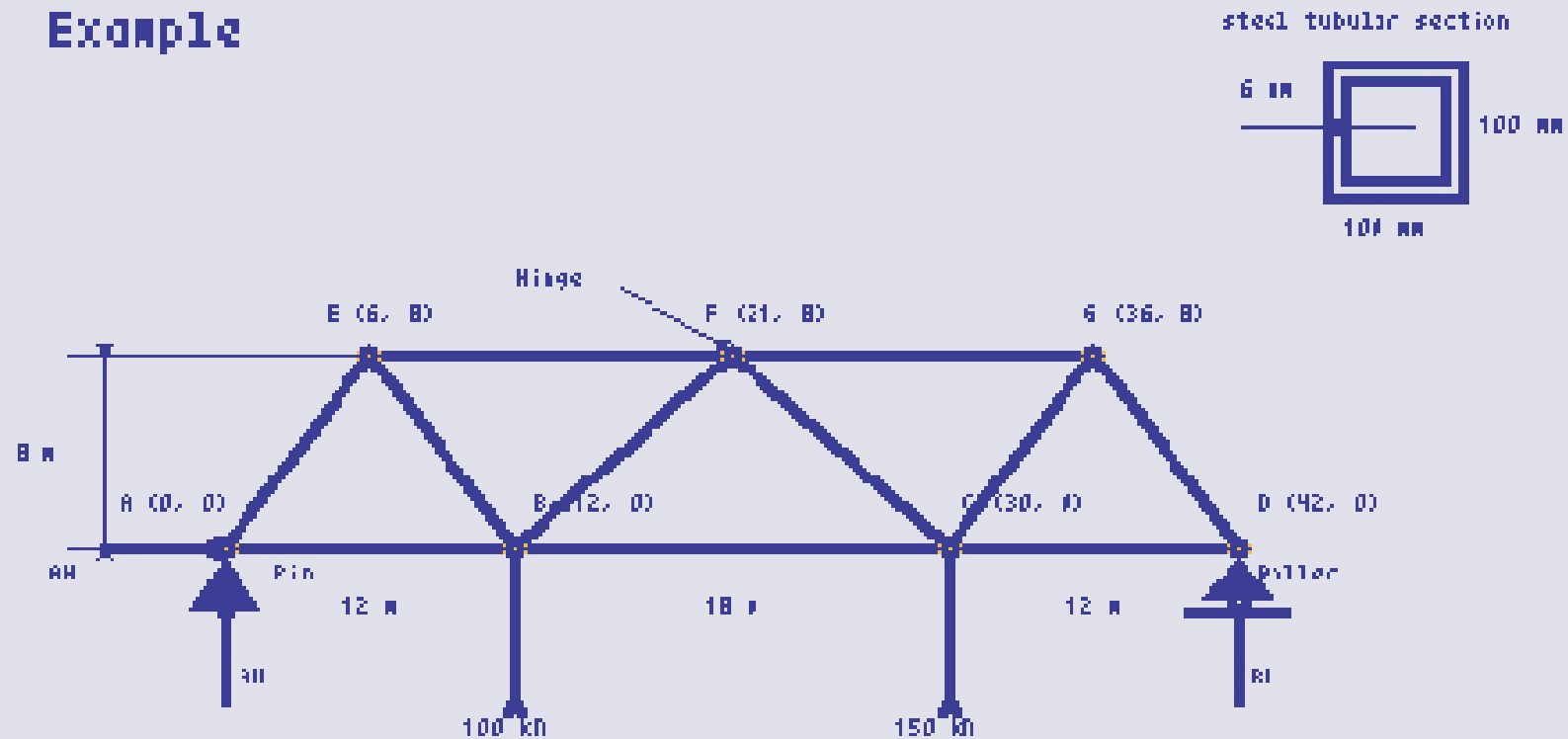


Part 4

Example 1: A two-dimensional truss analysis
program

A two-dimensional truss program

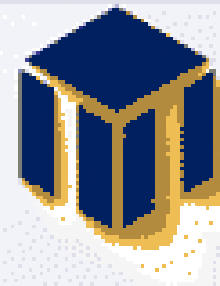
Example



Part 5

Python FEA libraries

Python FEA libraries



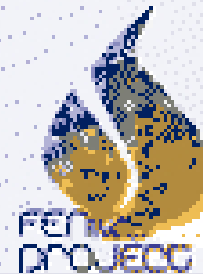
NextFEM

L'analisi a elementi finiti facile

concreteproperties

scikit-fem

sectionproperties



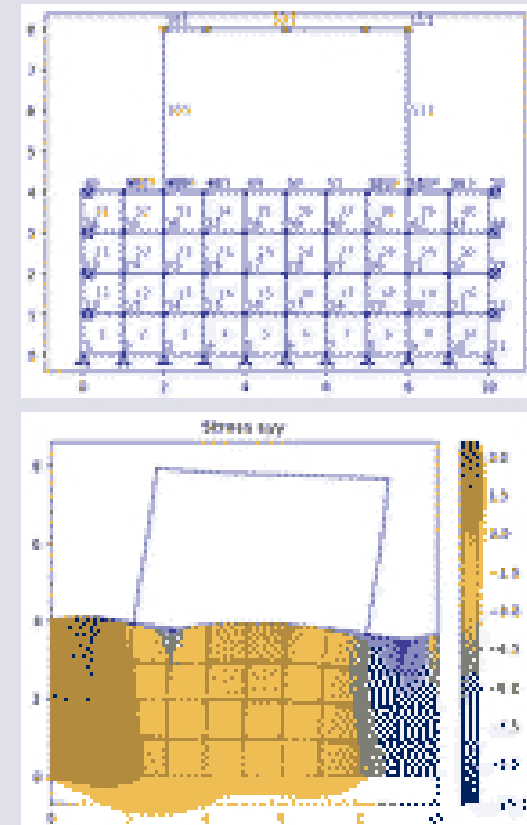
EasyFEA



The openseespy library

OpenSees (Open System for Earthquake Engineering Simulation) is a software framework for developing applications to simulate the performance of structural and geotechnical systems subjected to earthquakes, developed by the Pacific Earthquake Engineering Research (PEER) Center.

```
1 # Install the main library.
2 pip install openseespy
3
4 # Install results visualization engine.
5 pip install opsviz      # using matplotlib
6 pip install vfo         # using pyvista
```



Soil-structure interaction.



The openseespy library

Example

```
1 import openseespy.opensees as ops
2 ops.wipe()
3
4 # Create a two-dimensional model with two
5 # degrees of freedom only (translation x, y).
6 ops.model('basic', '-ndm', 2, '-ndf', 2)
7
8 # Create nodes.
9 ops.node(1, 0.0, 0.0)
10 ops.node(2, 4.0, 0.0)
11 ops.node(3, 0.0, 3.0)
12
13 # Create boundary conditions.
14 ops.fix(1, 1, 1)      # pinned support
15 ops.fix(2, 0, 1)      # roller support
16
17 # Create material for each member.
18 ops.uniaxialMaterial('Elastic', 1, 200e6)
19
20 # Create elements and assign material.
21 ops.element('Truss', 1, 1, 2, 0.01, 1)
22 ops.element('Truss', 2, 1, 3, 0.01, 1)
23 ops.element('Truss', 3, 2, 3, 0.01, 1)
```

```
1 # Create loads
2 ops.timeSeries('Linear', 1)
3 ops.pattern('Plain', 1, 1)
4 ops.load(3, 5.0, -10.0)      # point load
5
6 # Analysis options
7 ops.constraints('Plain')
8 ops.numberer('Plain')
9 ops.system('BandGeneral')
10 ops.algorithm('Linear')      # linear analysis
11 ops.analysis('Static')       # static analysis
12 ops.analyze(1)
13
14 # Extract results.
15 print(ops.nodeDisp(3))      # displacements
16 print(ops.nodeReaction(1))  # reactions
```

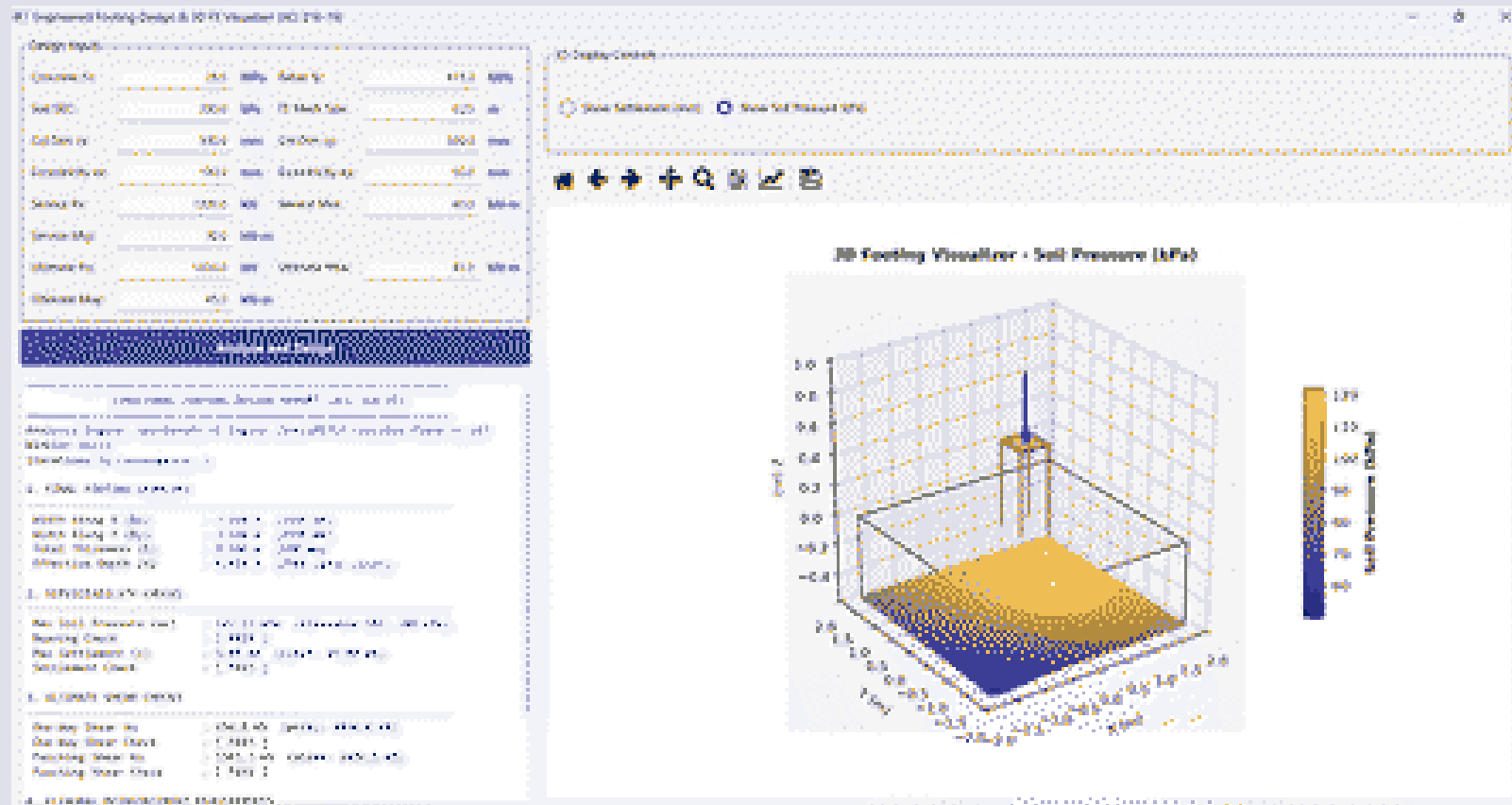


Part 6

Example 2: An isolated footing design program

Isolated footing program

Sample output



Isolated footing program

Prompt

Take the role of a structural engineer with several years of experience in the analysis, design and detailing of structures of various heights and lengths (horizontal/vertical).
Create a GUI program in Python using the PySide6 library that does the following.

Accept as inputs:

- (a) 28-day compressive strength of concrete f_c' ,
- (b) reinforcement yield strength f_y ,
- (c) soil bearing capacity SBC (kPa),
- (d) column dimensions along x and y directions (rectangular),
- (e) column eccentricity from center of footing along x and y directions,
- (f) factored service loads at column (P_s , M_{sx} , M_{sy} , V_{sx} , V_{sy} , T_s),
- (g) factored ultimate loads at column (P_u , M_{ux} , M_{uy} , V_{ux} , V_{uy} , T_u),
- (h) footing maximum finite element size.



Isolated footing program

Prompt

Accept as output:

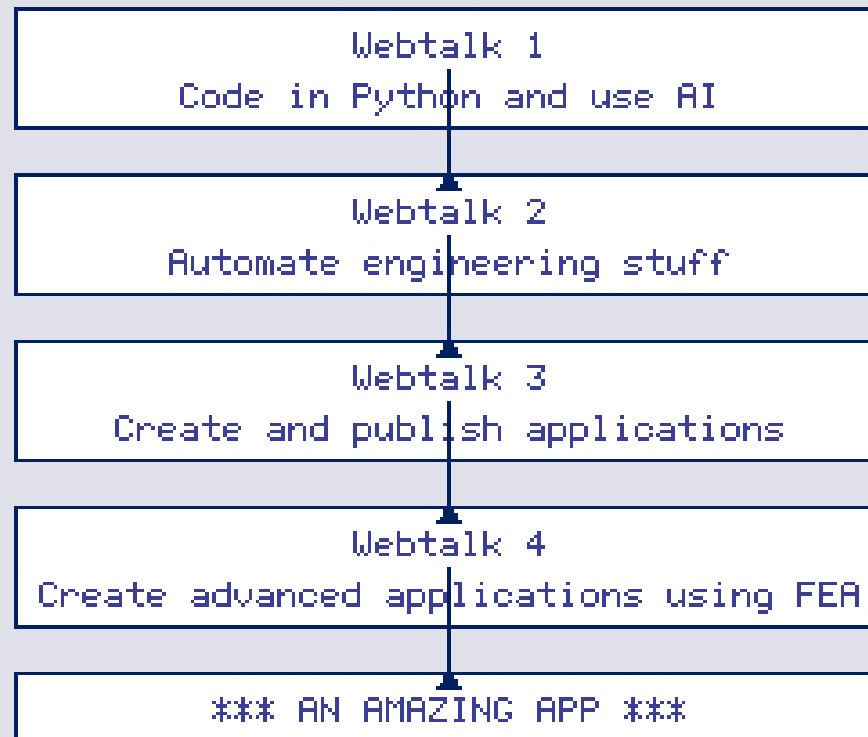
- (a) a three-dimensional visualization of the footing,
- (b) upon pressing [Analyze and Design], create a finite element model using OpenSeesPy library,
 - > take the subgrade modulus as 120 times the SBC
 - > use compression-only springs to model the soil
 - > the stiffness of horizontal springs is 10% of the stiffness of vertical springs,
- (c) report the following (display contour plots in three-dimensional visualization):
 - > settlement (in mm),
 - > soil pressure (in kPa),
- (d) design the footing based on NSCP 2015 provisions, increase sizes iteratively where needed:
 - > footing dimension in the x- and y- directions,
 - > footing thickness from one-way and two-way shear checks
 - > footing reinforcement from one-way moment checks in each direction.



Part 7

Next steps

Putting it all together



Commercial software do their best to cover all possible use cases by the engineer.

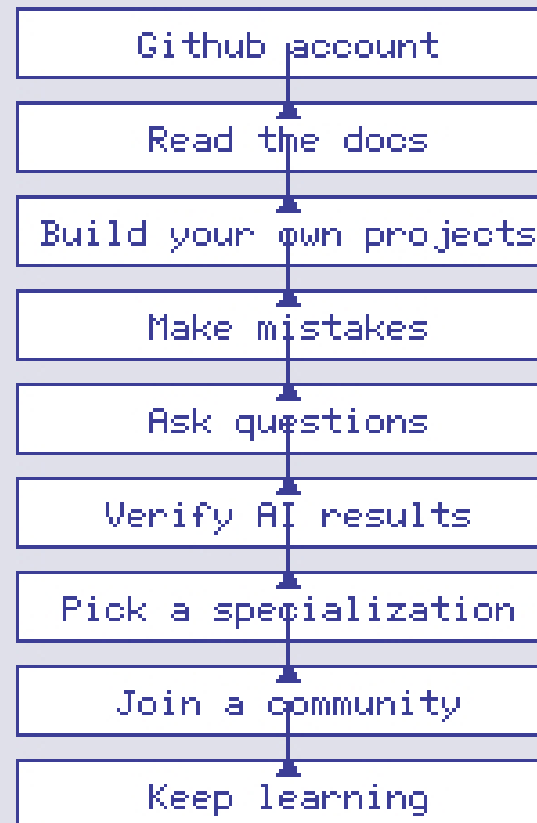
However, there will be use cases that are not covered by the software.

Programming is there to give the engineer the power to make his own.

From software user to software developer.



Next steps



Closing

This presentation was created using a heavily modified version of Pyxel slides, which I learned at PythonAsia 2026 from the presentation of Takayuki Shimizukawa. Special thanks to Shimizukawa-sensei for creating such an amazing tool!

Contact me

- * Email: jaydee.lucero@gmail.com
- * Facebook: facebook.com/jaydee.lucero
- * Linkedin:
linkedin.com/in/jaydee-lucero-977070200
- * Github: github.com/arisa-chan
- * Website: engr.jaydee.com

PDF and source code of presentation



Thank you very much!

Do you have any questions?